INSTITUTO NACIONAL DE APRENDIZAJE

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

Diseño de algoritmos

Sesión 22 · Paso de parámetros por valor y por dirección

UA2 · Programación estructurada | Módulo CSTI12010

Persona facilitadora

Giovanni Antonio Coto Calderón



Parámetro en C:
Es la variable que se define en la **declaración o firma** de una función. Funciona como un "recipiente" o espacio reservado que espera recibir un dato cuando la función sea ejecutada.

Argumento en C:
Es el valor real (o la variable con un valor) que se le **envía** a la función en el momento exacto en que se la llama o invoca.

Ejemplo rápido: Si define
void sumar(int a), **a** es el parámetro.
Si luego la llamas escribiendo
sumar(5);, el **5** es el argumento.

Paso por valor: la función recibe una copia

En C los argumentos se pasan **por valor**: el parámetro recibe una **copia** del dato. La función puede cambiar su copia, pero la **variable original no cambia**.

Copia

x = 5 se copia en el parámetro.

Protección

La función no altera main.

Para datos de entrada

Lo que la función solo lee.

Resultado con return

Un solo valor.

La copia cambia; el original no

valor.c

```c
#include <stdio.h>
void duplicar(int x) {  //x es el parámetro (contenedor)
    x = x * 2;                  // cambia la COPIA
    printf("dentro: x = %d\n", x);
}
int main(void) {
    int numero = 5;

     // Al llamar a la función, 'numero' es el ARGUMENTO (el valor real enviado)
    duplicar(numero);
    printf("main: numero = %d\n", numero);
    return 0;
}
```

Salida real (gcc)

```
dentro: x = 10
main: numero = 5
```

SABER 2

Direcciones de memoria y punteros



Cada variable vive en una **dirección** de memoria. **&x** obtiene esa dirección. Un **puntero** (**int *p**) es una variable que guarda una dirección, y ***p** es el valor que hay en ella.



&x

Dirección de x.



int *p

Puntero a un int.



p = &x

p apunta a x.



***p = 21**

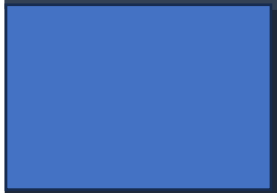
Cambia x a través de p.

Leer y cambiar a través de un puntero

puntero.c

```c
#include <stdio.h>
int main(void) {
    int edad = 20;
    int *p = &edad;             // p guarda la direccion
    printf("edad = %d\n", edad);
    printf("*p   = %d\n", *p);  // lee lo que hay alli
    *p = 21;                    // cambia edad a traves de p
    printf("edad = %d\n", edad);
    return 0;
}
```

Salida real (gcc)



Intercambio (sesión 6) con punteros

La función recibe **direcciones** (**int *a**); la llamada envía **&x** y **&y**. Dentro, ***a** y ***b** son las variables de main.

intercambio.c

```c
#include <stdio.h>
void intercambiar(int *a, int *b) {
    int aux = *a;
    *a = *b;
    *b = aux;
}
int main(void) {
    int x = 5, y = 9;
    printf("Antes:   x = %d  y = %d\n", x, y);
    intercambiar(&x, &y);       // envia direcciones
    printf("Despues: x = %d  y = %d\n", x, y);
    return 0;
}
```

Salida real (gcc)

```
Antes:   x = 5  y = 9
Despues: x = 9  y = 5
```

CUIDADO

Por valor no intercambia

Con parámetros **int a, int b** la función intercambia **copias**: compila sin advertencias, pero **x e y no cambian**.

Por valor

Salida: Despues: x = 5 y = 9

```
void intercambiar(int a, int b) {
    int aux = a;
    a = b;
    b = aux;
}
intercambiar(x, y);
// Despues: x = 5  y = 9
```

Por dirección

Salida: Despues: x = 9 y = 5

```
void intercambiar(int *a, int *b) {
    int aux = *a;
    *a = *b;
    *b = aux;
}
intercambiar(&x, &y);
// Despues: x = 9  y = 5
```

Devolver dos resultados: DivisionEntera

return entrega **un** valor. Para devolver **cociente y residuo** (DivisionEntera, sesión 9), la función los **escribe** en variables de main recibidas por dirección.

dividir.c

```c
#include <stdio.h>
void dividir(int a, int b, int *cociente, int *residuo) {
    *cociente = a / b;
    *residuo = a % b;
}
int main(void) {
    int c, r;
    dividir(17, 5, &c, &r);      // dos resultados
    printf("17 / 5 = %d residuo %d\n", c, r);
    return 0;
}
```

Salida real (gcc)

```
17 / 5 = 3 residuo 2
```

Arreglos como parámetros

Un arreglo se pasa por su **nombre**, que ya es una **dirección**: la función trabaja sobre el **original**. Se envía también su **tamaño**. **const** impide modificarlo.

arreglos.c (fragmento)

```c
#define N 5
double promedio(const double notas[], int n) { //a palabra clave const sirve para proteger los datos del arreglo.
    double suma = 0;
    int i;
    for (i = 0; i < n; i++) { suma += notas[i]; }
    return suma / n;
}
void aplicarCurva(double notas[], int n, double puntos) {
    int i;
    for (i = 0; i < n; i++) { notas[i] += puntos; }
}
int main(void) {
    double notas[N] = {80, 75, 90, 65, 100};
    printf("Promedio: %.2f\n", promedio(notas, N));
    aplicarCurva(notas, N, 2);  // cambia el original
    printf("Con curva: %.2f\n", promedio(notas, N));
```

Salida real (gcc)

```
Promedio: 82.00
Con curva: 84.00
```

Cadenas como parámetros

Una cadena es un arreglo de char: se pasa igual (**const char texto[]**). No hace falta el tamaño: el **'\0'** marca el final. Se integra con **ctype.h**; main llama **contarVocales("Instituto Nacional")**.

vocales.c (fragmento)

```
int contarVocales(const char texto[]) {
    int i, total = 0;
    char c;
    for (i = 0; texto[i] != '\0'; i++) {
        c = tolower(texto[i]);
        if (c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u') {
            total++;
        }
    }
    return total;
}
```

Salida real (gcc)

```
Vocales: 8
```

Estructura de un programa modular

Orden	Sección	Ejemplo
1	Comentario inicial: qué hace y resultado esperado	`// sistema_notas.c - Registro de notas …`
2	Bibliotecas	`#include <stdio.h>  #include <string.h>`
3	Constantes	`#define MAX 16`
4	Prototipos (con comentario)	`void agregarNota(double notas[], int *cantidad);`
5	main: el índice del programa	`menú con do … while y switch que llama funciones`
6	Definiciones, una tarea cada una	`double leerNota(void) { … }`

Así deben verse los programas de la **Actividad de comprobación 3**: comentados, compilando con **-Wall -Wextra** sin advertencias y con commits de avance.

RESUMEN

¿Por valor o por dirección?



Por valor

La función recibe una copia

- La función solo LEE el dato.
- Un resultado: con return.
- double calcularIVA(double monto)
- Llamada: calcularIVA(precio)



Por dirección

La función recibe un puntero

- La función CAMBIA la variable.
- Varios resultados.
- void dividir(…, int *c, int *r)
- Llamada: dividir(a, b, &c, &r)